RAPPORT TECHNIQUE

RSA File Secure

Conception, architecture, fonctionnement et limites du projet de chiffrement

Application web Flask de chiffrement et de messagerie

Python · Flask · SQLite · HTML/CSS/JavaScript · RSA

Rapport etabli a partir du code source, des templates et des tests.
Etat du depot examine le 13 octobre 2026

Sommaire

Sommaire 2
1. Synthese du projet 3
2. Objectifs et perimetre 3
2.1 Objectifs fonctionnels . . . 3
2.2 Perimetre technique . . . 3
3. Architecture logicielle 3
3.1 Composants . . . 4
3.2 Routes importantes . . . 4
4. Fonctionnalites et parcours utilisateur 4
4.1 Comptes et profils . . . 4
4.2 Generation et association des cles . . . 4
4.3 Contacts et conversations . . . 4
4.4 Messages et decryptage dans le chat . . . 5
4.5 Fichiers et expiration . . . 5
4.6 Outils generiques . . . 5
5. Modele de donnees et stockage 5
5.1 Stockage de fichiers . . . 5
6. Fonctionnement cryptographique 5
6.1 Generation RSA . . . 5
6.2 RSA par blocs . . . 6
6.3 Format .rsa . . . 6
7. Securite : garanties et limites 6
7.1 Mesures presentes . . . 6
7.2 Limites cryptographiques . . . 6
7.3 Limites de deploiement . . . 6
8. Tests et verification 7
9. Installation et lancement 7
10. Structure du depot 7
11. Ameliorations recommandees 8
12. Conclusion 8

1. Synthese du projet

RSA File Secure est une application web en francais qui permet de creer des comptes, gerer des contacts et echanger des messages et fichiers chiffres avec RSA. Elle associe une interface Flask/Jinja, une base SQLite locale et une implementation pedagogique du RSA en Python.

Les messages et pieces jointes sont chiffres avec la cle publique du destinataire. Les messages sont conserves en base sous forme chiffree ; les fichiers sont conserves temporairement dans un dossier local. Pour lire un message dans le chat, le destinataire importe sa cle privee : le serveur effectue le decryptage et renvoie le texte au navigateur, sans enregistrer le texte clair.

Aspect	Fonctionnement observe
Comptes et profils	Inscription, connexion, profil, avatar, biographie et session Flask.
Cles	Generation RSA 2048 bits pour un compte connecte ; cle publique stockee dans le profil, cle privee a telecharger.
Messages	Conversations entre contacts acceptes ; payload chiffre en base ; decryptage a la demande dans le chat.
Fichiers	Chiffrement/dechiffrement au format .rsa et transferts a duree configurable.
Stockage	SQLite pour comptes, contacts, messages et metadonnees ; fichiers dans instance/.
Tests	26 tests automatises passent dans l'environnement du depot.

Point d'attention
Le projet applique RSA brut par blocs, sans OAEP ni chiffrement hybride. C'est un prototype pedagogique et non une solution adaptee a des secrets en production.

2. Objectifs et perimetre

2.1 Objectifs fonctionnels

- Creer des comptes, gerer des profils et etablir des relations de contact acceptes.
- Generer une paire RSA et associer la cle publique au compte sans enregistrer la cle privee dans la base.
- Envoyer des messages textes et des fichiers chiffres pour le destinataire.
- Proposer un decryptage de fichier et un decryptage a la demande dans les conversations.
- Gerer une reception, un historique et l'expiration des transferts de fichiers.

2.2 Perimetre technique

L'application est un monolithe Flask. Les routes HTTP, la configuration et la coordination sont dans `app.py`. La logique RSA est separee dans `rsa_core/`, le schema et les fonctions de stockage dans `storage.py`, l'interface dans les templates Jinja, CSS et JavaScript.

3. Architecture logicielle

Le flux principal relie le navigateur, Flask et SQLite. A l'envoi, le serveur chiffre le texte avec la cle publique du contact et stocke le binaire chiffre. A la lecture, le navigateur transmet le fichier de cle privee au endpoint de decryptage puis recoit le texte en JSON.

Navigateur Jinja, HTML, CSS, JavaScript	**Application Flask** Routes, sessions, coordination RSA	**SQLite et fichiers** Payloads chiffres et metadonnees
Texte ou fichier + cle publique	Chiffrement et preparation du format .rsa	Payload en base ou fichier chiffre sur disque
Importe sa cle privee pour lire	Decryptage puis reponse JSON/fichier	Pas d'enregistrement du texte clair du chat

3.1 Composants

Composant	Responsabilite
app.py	Routes Flask, authentification, generation de cles, conversations et transferts.
storage.py	Initialisation SQLite, connexions, dates, expiration, paire de contacts.
rsa_core/primes.py	Generation de nombres premiers probables avec secrets et Miller-Rabin.
rsa_core/keys.py	Structures de cles, conversion JSON, generation de paires.
rsa_core/cipher.py	RSA modulaire, traitement par blocs, CRT et verification du digest.
rsa_core/fileformat.py	Lecture et ecriture de l'en-tete binaire .rsa.
templates/ et static/	Pages Jinja, styles CSS et interactions JavaScript.
tests/	Tests unitaires cryptographiques et tests HTTP Flask.

3.2 Routes importantes

Domaine	Routes	Role
Compte	/register, /login, /logout, /profile	Inscription, session et profil.
Contacts	/contacts, /contacts//	Demandes et statuts de relation.
Cles	/keys, /keys/generate, /profile/key	Generation protege et publication/import de cle publique.
Messagerie	/messages, /messages/	Liste des chats, envoi de texte et fichier.
Decryptage chat	/messages//decrypt	Import temporaire de cle et retour du texte clair.
Transferts	/send, /inbox, /history, /transfers//download	Envoi, reception, suivi et telechargement.
Outils RSA	/encrypt, /decrypt	Chiffrement et decryptage generiques.

4. Fonctionnalites et parcours utilisateur

4.1 Comptes et profils

Le pseudo doit avoir 3 a 32 caracteres alphanumeriques, tirets ou underscores ; le mot de passe a au moins 8 caracteres. Werkzeug calcule un hash PBKDF2-SHA256. La connexion place l'identifiant dans la session Flask. Le profil accepte nom affiche, biographie limitee a 280 caracteres et avatar aux extensions JPG/JPEG/PNG/WebP.

4.2 Generation et association des cles

La page Cles est reservee aux comptes connectes. Le serveur genere une paire de 2048 bits, enregistre le JSON public dans `users.public_key` du compte courant et renvoie les donnees au navigateur. JavaScript permet de telecharger `cle_privee.json`. La cle privee n'est pas enregistree dans la base. Generer une nouvelle paire remplace la cle publique ; il faut conserver les anciennes cles privees pour les anciens contenus.

Sauvegarde indispensable
Si le fichier prive est perdu, le serveur ne peut pas reconstituer la cle permettant de lire les anciens contenus.

4.3 Contacts et conversations

Les contacts sont stockes par paire d'identifiants normalisee. Les statuts sont pending, accepted, rejected et blocked. Seuls les contacts acceptes accedent aux echanges. Un message saisi dans le chat est limite a 5 000 caracteres.

4.4 Messages et decryptage dans le chat

Le message texte UTF-8 est chiffre comme un contenu logique `message.txt` et stocke dans `messages.encrypted_payload`. Pour un message entrant, le destinataire choisit un fichier JSON de cle privee et clique « Déchiffrer le message ». Le serveur verifie le destinataire, parse la cle, dechiffre et renvoie le texte UTF-8 en JSON. Le texte clair n'est pas ecrit en base.

Ce flux n'est pas un decryptage entierement local : le fichier prive transite vers le serveur dans la requete, et le texte clair revient depuis le serveur au navigateur. L'administrateur ou un serveur compromis peut donc potentiellement voir la cle pendant le traitement.

4.5 Fichiers et expiration

L'expediteur choisit un contact avec cle publique, un fichier ou texte et une echeance : 24 heures, 7 jours ou suppression apres le premier telechargement. Le contenu est chiffre, sauvegarde sous un nom aleatoire dans `instance/encrypted_uploads` puis reference dans la table transfers. Le destinataire telecharge le fichier .rsa avant de le dechiffrer. Le nettoyage supprime les fichiers expires.

4.6 Outils generiques

Les pages Chiffrer/Dechiffrer offrent des formulaires asynchrones : selection du fichier et import du JSON de cle, avec telechargement du resultat. Le decryptage accepte une cle privee importe ou ses composantes. Flask limite la taille d'une requete a 8 Mio et retourne une erreur 413 si cette limite est depassee.

5. Modele de donnees et stockage

La base effectivement configuree est `instance/rsa_file_secure.db`, initialisee au demarrage. Le schema cree les tables manquantes et ajoute certaines colonnes lors des evolutions. Le fichier `rsa_platform.db` a la racine n'est pas le chemin configure par l'application.

Table	Champs principaux	Remarques
users	id, username, password_hash, public_key, profil, created_at	Cle publique associee ; pas de colonne privee.
contacts	user_a, user_b, requested_by, status, dates	Paire composite ; statuts contraints.
messages	id, sender_id, recipient_id, encrypted_payload, dates	Corps conserve sous forme binaire chiffree.
transfers	expediteur, destinataire, nom, taille, echeance, etat	Metadonnees du fichier chiffre conserve sur disque.

5.1 Stockage de fichiers

- Les fichiers de transfert chiffres sont dans `instance/encrypted_uploads/`.
- Les avatars sont dans `instance/avatars/`.
- La base et les fichiers locaux sont separes du depot de code dans le dossier instance/.

SQLite est adapte au prototype local. Un usage concurrent ou distribue demande une base serveur, des migrations versionnees et une strategie de sauvegarde et de retention.

6. Fonctionnement cryptographique

6.1 Generation RSA

Le programme genere deux premiers p et q, calcule $n = p \times q$ et $\varphi(n) = (p-1)(q-1)$, puis utilise $e = 65537$. La cle privee est d, inverse modulaire de e modulo $\varphi(n)$. Pour une cle de 2048 bits, les facteurs generes font environ 1024 bits chacun. Les candidats sont tires avec `secrets` et testes par Miller–Rabin pendant 40 tours, apres filtrage de petits premiers.

Cette implementation permet d'etudier les principes. Un projet protegeant de vraies donnees devrait s'appuyer sur une bibliotheque cryptographique maintenue et auditee.

6.2 RSA par blocs

Chaque bloc clair est converti en entier m inferieur a n. Le chiffrement calcule $c = m^e \bmod n$ et le decryptage $m = c^d \bmod n$. Si p et q sont inclus dans la cle privee, le calcul utilise le theoreme des restes chinois (CRT).

Module RSA	Bloc clair	Bloc chiffre	Utilisation
1024 bits	127 octets	128 octets	Plusieurs tests HTTP.
2048 bits	255 octets	256 octets	Generation actuelle via l'interface.

Les blocs chiffres ont une taille fixe. L'en-tete preserve la taille d'origine et permet de reconstruire exactement le dernier bloc, y compris les octets nuls en tete.

6.3 Format .rsa

La version 1 contient la signature RSAF, la version, la longueur et le nom UTF-8, la taille originale, les tailles des blocs clair/chiffre, le SHA-256 du contenu puis les blocs chiffres.

Le SHA-256 aide a detecter une cle incorrecte ou une corruption accidentelle, mais il n'est pas authentifie : ce n'est ni une signature, ni une preuve d'origine, et un attaquant capable de modifier le contenu peut recalculer le digest.

7. Securite : garanties et limites

7.1 Mesures presentes

- Les mots de passe sont stockes sous forme de hash PBKDF2-SHA256 via Werkzeug.
- Les fonctions de compte, contacts, messagerie, transferts et generation de cles sont protegees par session.
- Le decryptage d'un message verifie que l'utilisateur connecte est bien son destinataire.
- La plupart des acces SQL utilisent des parametres lies ; les noms de transfert internes sont aleatoires.
- La cle privee n'a pas de champ dans le schema ; les messages restent stockes chiffres.

7.2 Limites cryptographiques

- **RSA sans padding :** le RSA brut par blocs est deterministe et ne fournit pas les garanties d'un chiffrement moderne.
- **Pas de schema hybride :** RSA chiffre directement chaque bloc. En pratique, RSA-OAEP protege une cle aleatoire et AES-GCM chiffre les donnees.
- **Pas d'authentification de cle :** aucune empreinte ou canal de confiance ne confirme qu'une cle publique appartient vraiment au contact.
- **Digest non authentifie :** SHA-256 detecte certaines erreurs mais ne garantit ni l'integrite contre un attaquant ni l'origine.
- **Metadonnees visibles :** nom de fichier, taille, version, tailles des blocs et digest figurent dans l'en-tete .rsa.

7.3 Limites de deploiement

- Une cle Flask de developpement est utilisee par defaut si RSA_SECRET_KEY est absent ; elle doit etre remplacee en production.
- Le lancement direct execute Flask avec `debug=True` ; ne pas exposer ce serveur de developpement sur un reseau public.
- La cle privee du decryptage de chat transite au serveur. HTTPS/TLS est indispensable en transit, sans pour autant proteger contre le serveur lui-meme.
- La taille des requetes est limitee a 8 Mio et les messages a 5 000 caracteres ; RSA par blocs est couteux sur les contenus volumineux.
- La rotation remplace la cle publique du compte ; il n'existe pas de versionnement de cle ni de chiffrement local de bout en bout.

Conclusion securite
Le projet convient a un exercice de cryptographie. Ne pas l'utiliser pour des donnees sensibles en production sans revoir le protocole RSA, le decryptage du chat et la configuration serveur.

8. Tests et verification

La suite a ete executee avec pytest : **26 tests passes**. Cela verifie les scenarios couverts mais ne constitue pas un audit de securite ni une preuve de robustesse cryptographique.

Fichier	Couverture
test_primes.py	Petits premiers, nombres de Carmichael, premier connu et taille du premier genere.
test_keys.py	Exemple RSA p=61, q=53 ; cles avec/sans CRT ; JSON et entrees invalides.
test_cipher.py	Tailles de blocs, donnees binaires, cle incorrecte, corruption et fichier invalide.
test_app.py	Routes HTTP, limites, generation protegee et rattachee au compte, decryptage chat et cle incorrecte.

Le test de generation verifie par remplacement du generateur que la route demande 2048 bits et stocke la cle publique. Les tests ne couvrent pas tous les navigateurs, les attaques reseau, TLS, ni une charge de production.

9. Installation et lancement

Depuis le dossier du projet sous Windows PowerShell :

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

Le site est accessible sur `http://127.0.0.1:5000`. Pour lancer les tests :

```
.\.venv\Scripts\python.exe -m pytest -q
```

La base SQLite et les dossiers de donnees sont crees sous instance/. Les dependances du projet sont Flask et pytest ; le noyau RSA s’appuie sur Python.

10. Structure du depot

`app.py` — routes et configuration Flask
`storage.py` — schema SQLite et stockage
`rsa_core/primes.py` — generation probabiliste de premiers
`rsa_core/keys.py` — modeles, parsing et paires de cles
`rsa_core/cipher.py` — traitement RSA par blocs
`rsa_core/fileformat.py` — format binaire .rsa
`rsa_core/errors.py` — erreurs metier
`templates/` — interface Jinja
`static/css/style.css` — styles
`static/js/main.js` — interactions web
`tests/` — tests unitaires et HTTP
`instance/` — base et donnees locales d’execution

11. Ameliorations recommandees

Priorite	Amelioration	Motif
1 — critique	Passer a RSA-OAEP pour une cle AES aleatoire, AES-GCM pour les donnees et une bibliotheque crypto eprouvee.	Confidentialite, authentification, performance et resistance aux attaques.
2 — critique	Dechiffrer dans le navigateur ; ne jamais transmettre la cle privee au serveur.	Reduire l'exposition et viser un vrai chiffrement de bout en bout.
3 — elevee	Supprimer debug=True et la cle par defaut ; imposer TLS, cle de session secrete et cookies Secure/HttpOnly/SameSite.	Durcir sessions et configuration d'exploitation.
4 — elevee	Authentifier les cles publiques, verifier leur empreinte et gerer rotation/version.	Eviter la substitution de cle et conserver l'acces historique.
5 — elevee	Ajouter CSRF, limites de debit, validation stricte et tests d'autorisation.	Robustesse face aux abus et requetes malveillantes.
6 — moyenne	Migrations versionnees, sauvegardes, politique de retention, supervision et tests navigateur/performance.	Fiabilite et evolution du produit.

12. Conclusion

RSA File Secure rassemble comptes, contacts, generation de cles, messagerie, partage de fichiers et stockage SQLite autour d'un module RSA pedagogique. La separation entre routes, noyau cryptographique et templates rend le fonctionnement lisible, et les 26 tests confirment les principaux cas automatises.

La principale reserve est la difference entre objectif pedagogique et securite operationnelle. Le RSA brut par blocs, l'absence d'authentification des cles et l'envoi de la cle privee au serveur pour lire un message empechent de qualifier l'application de messagerie confidentielle de bout en bout. La priorite est de remplacer le protocole cryptographique puis de deplacer tout decryptage dans le navigateur et de securiser le deploiement.

Fin du rapport — RSA File Secure